E-Commerce Recommendation Engine: A Hybrid Content-Based and Collaborative Ranking Framework

Kontham Vivek
Scholar, M.Tech
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
2601050035@kluniversity.in

Raju Anitha
Professor
Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation
Vaddeswaram, Guntur, Andhra Pradesh, India
anitharaju@kluniversity.in

***Abstract*—Online catalogs contain far more products than a shopper can browse, so a recommender must rank items that fit both long-term interests and the current session. This paper presents a hybrid ranking framework that combines content-based cosine similarity, item-based collaborative filtering, time-decayed popularity, and a contextual signal in one weighted score. Weights are renormalized per request when a component is unavailable, so new and sparse-history users fall back to popularity without treating missing evidence as a zero score. The paper specifies the data contract, eligibility rules, a chronological evaluation protocol, and a reference implementation. The framework is evaluated in a controlled simulation of 2,000 users and 600 items with preference drift, light users, and new users, averaged over five random seeds. The tuned hybrid reaches Recall@10 of 0.174 ± 0.007, against 0.090 for popularity, 0.080 for content-only, and 0.172 for item-based collaborative filtering. Its overall advantage over item-based filtering is within one standard deviation, although it is higher for sparse-history users. Because the data are synthetic, the results validate the pipeline and its fallback behavior, not performance in a real store.**

***Index Terms*—E-commerce, recommender systems, hybrid ranking, content-based filtering, collaborative filtering, cold start, time decay, Precision@K, Recall@K.**

I. Introduction

E-commerce interfaces expose shoppers to catalogs of thousands or millions of products. Search, category browsing, and curated lists remain useful, but they do not reflect an individual shopper's interests or current intent. Recommender systems address this information overload by estimating which products are likely to be relevant and presenting them as a ranked list. A systematic review of e-commerce recommendation identifies content-based, collaborative, demographic, knowledge-based, and hybrid approaches, together with open problems of scalability, sparsity, and evaluation [1]. Deep-learning methods add further model families but inherit the same cold-start and sparsity issues [2].

Implicit feedback is central in retail because explicit ratings are scarce. Product views and searches indicate interest but may be exploratory; cart additions are stronger intent signals; purchases are strong but do not always imply a wish to buy the same item again. These events are noisy evidence, so event weights and time decay should be configured and validated rather than assumed.

No single signal is sufficient. Content similarity can recommend new products but tends to over-specialize; collaborative signals reveal latent relations but need dense histories; popularity is robust but concentrates exposure on a few items. A practical engine must therefore combine signals and behave sensibly when some are missing. This paper documents such an engine as a transparent baseline with a clear data contract, modular scoring, deterministic eligibility filtering, and a leakage-free offline evaluation.

The contributions are as follows.

1) A layered architecture that separates event ingestion, feature preparation, candidate generation, ranking, eligibility filtering, and evaluation.
2) A hybrid score with per-request weight renormalization and a deterministic popularity fallback, so that missing evidence is never scored as dislike.
3) A chronological evaluation protocol with cohort reporting for new, sparse-history, and returning users.
4) A reproducible simulation study with measured results, including a negative finding: the hybrid is not clearly better than item-based filtering overall.

Section II reviews related work, Section III formulates the problem, Section IV describes the architecture, Section V gives the method, Section VI reports the experiments, and Sections VII and VIII discuss limitations and conclude.

II. Related Work

A. Recommendation Families

Item-to-item collaborative filtering scales to large catalogs by precomputing item similarities and was deployed at Amazon [3]; its neighborhood formulation is analyzed in [4]. Matrix factorization models learn latent user and item factors and became a standard for rating prediction [5]. Content-based methods instead compare item descriptions with a user profile, which allows new items to be recommended but may over-specialize around items already seen.

B. Implicit Feedback

Hu et al. treat implicit events as confidence-weighted positive observations rather than ratings [6], and Bayesian personalized ranking optimizes pairwise orderings from implicit data [7]. The event-strength model in this paper follows the same principle of weighting event types, but it keeps the weights explicit and configurable.

C. Hybrid Recommendation

Burke surveys hybridization strategies, including weighted, switching, and cascade designs, and notes that hybrids are justified by complementary strengths rather than guaranteed accuracy gains [8]. The framework here is a weighted hybrid with a switching rule for missing components.

D. Cold Start

A new user with no history and a new item with no interactions are distinct cases. Item-based stereotypes [9] and deep hybrid item models [10] address the item side using

metadata. For new users, category-level popularity and current context provide a safe fallback; the proposed engine uses this fallback instead of treating absent evidence as negative preference.

E. Retrieval and Ranking

Large systems separate candidate retrieval from ranking: retrieval returns a manageable set with good recall, and ranking orders it with richer features [11]. Recent e-commerce work combines collaborative filtering, popularity, Bayesian personalized ranking, and learning-to-rank [12], and a recent review summarizes the broader role of artificial intelligence in e-commerce recommendation [13]. The present work follows the two-stage pattern but specifies a lightweight, interpretable baseline; it does not reproduce those systems or their reported results.

III. Problem Formulation

Let I be the catalog, U the set of users or sessions, and E the set of timestamped events. For a request from user u with context q, let H_u be the history available at request time and $I_q \subseteq I$ the eligible products (in stock, permitted, and not already excluded). The service returns an ordered list $L_K(u,q)$ of at most K distinct products from I_q. The objective is to place relevant products early while satisfying eligibility constraints. Because relevance is not observable for every product, held-out interactions serve as an offline proxy, and the resulting metrics must be interpreted cautiously.

The design is guided by five testable requirements: (R1) malformed catalog rows are rejected or quarantined; (R2) identical inputs produce identical rankings; (R3) no ineligible product appears in the output; (R4) an empty or sparse history triggers a documented fallback; and (R5) every ranking can be traced to its score components. Three research questions structure the evaluation: (RQ1) how can metadata and interaction evidence be combined without requiring long histories; (RQ2) how should the engine behave when a score component is unavailable; and (RQ3) what offline evidence is needed before claiming that a hybrid beats simpler baselines?

IV. System Architecture

Fig. 1 shows the logical architecture. The storefront sends a request with a user or session key and context. The service validates inputs, builds features from the catalog and event store, generates candidates from three sources, scores them, filters ineligible items, and returns a top-K list. Impressions and later feedback are logged so that offline evaluation can use only information available at recommendation time.

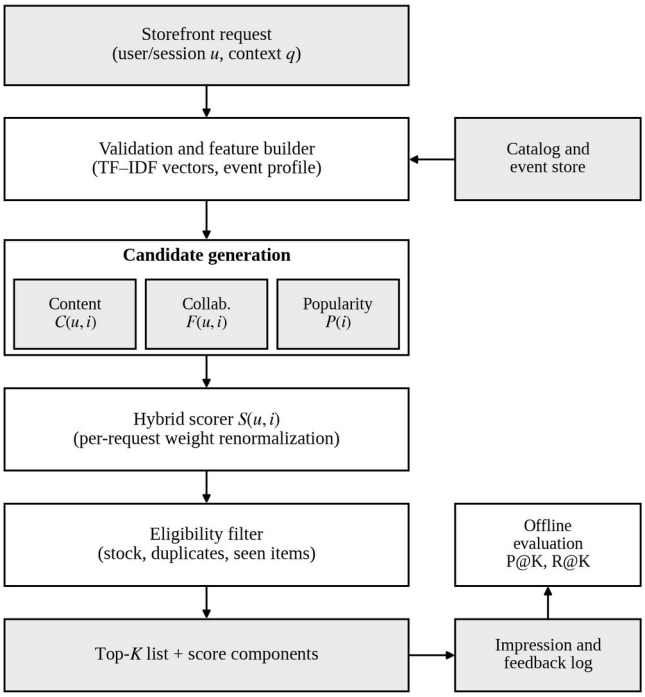


Fig. 1. Architecture of the recommendation engine. Shaded boxes are data stores, inputs, and outputs.

The stages are intentionally separable. Catalog ingestion validates identifiers, categories, descriptions, prices, and availability. Event ingestion records an event identifier, a pseudonymous user or session key, a product identifier, an event type, and a timestamp, which enables deduplication and chronological evaluation. Candidate generation limits the number of products that need detailed scoring. Eligibility filtering is explicit so that stale inventory or business exclusions cannot leak into the output. A missing value (for example, an unknown category) is represented as missing, not as zero.

Logging stores only the identifiers needed for evaluation, access to event histories is restricted, and retention and deletion follow the application's privacy policy.

V. Methodology

A. Item Representation

Each product i has a feature vector x_i built from lowercased, normalized text (title, category, brand, description) using TF–IDF weighting [14], concatenated with scaled categorical and numeric attributes. Vocabulary and statistics are fitted on the training period only, since fitting on test data leaks information. Empty descriptions and unknown categories are handled explicitly.

B. Event Strength and User Profile

Let t_0 be the request time and E_{ui} the events of user u on item i before t_0. The interaction strength applies an exponential time decay [15] to event-type weights:

$$r_{ui} = \sum_{e \in E_{ui}} a_{\text{type}(e)} \exp\left[-\lambda\,(t_0 - t_e)\right] \tag{1}$$

where a_{type} is the weight of an event type (view, cart, purchase) and λ is the decay rate. The user profile is the strength-weighted sum of the vectors of interacted items:

$$p_u = \sum_i r_{ui}\, x_i \tag{2}$$

C. Component Scores

The content score is the cosine similarity between the profile and a candidate, defined when both vectors have nonzero norm:

$$C(u,i) = \frac{p_u \cdot x_i}{\|p_u\| \, \|x_i\|} \tag{3}$$

The collaborative score is item-based. Similarity between items i and j is the cosine between their columns $r_{\cdot i}$ and $r_{\cdot j}$ of the strength matrix, and the score of a candidate is the similarity-weighted strength of the user's other items:

$$s_{ij} = \frac{r_{\cdot i} \cdot r_{\cdot j}}{\|r_{\cdot i}\| \, \|r_{\cdot j}\|} \tag{4}$$

$$F(u,i) = \sum_{j \in H_u} r_{uj} \, s_{ji} \tag{5}$$

The popularity score $P(i)$ is the time-decayed count of cart and purchase events for item i in the training period. The context score $X(u,i)$ equals 1 when the category of i matches the category of the user's most recent event and 0 otherwise.

D. Normalization, Availability, and Hybrid Score

Components have different scales, so each is min–max normalized to [0,1] over the eligible candidates of a request. A component is *available* only when its evidence exists (Table I). Let $a_k \in \{0,1\}$ indicate availability and $w_k \geq 0$ be the configured weight of component $k \in \{C, F, P, X\}$. The effective weights are renormalized once per request, so that all candidates of that request compete under the same weights:

TABLE I
COMPONENT AVAILABILITY BY USER STATE

User state (training events)	Active components
None (new user)	P only (popularity fallback)
1–2 events	C, P, X
3 or more events	C, F, P, X

$$\omega_k = \frac{a_k \, w_k}{\sum_l a_l \, w_l} \tag{6}$$

$$S(u,i) = \omega_C \, C(u,i) + \omega_F \, F(u,i) + \omega_P \, P(i) + \omega_X \, X(u,i) \tag{7}$$

If the denominator of (6) is zero, the engine ranks by popularity alone. Candidates are sorted by descending $S(u,i)$ with the product identifier as a deterministic tie-breaker.

Worked example (synthetic). Let $p = [1, 2, 0]$ and $x = [1, 1, 1]$. Then $p \cdot x = 3$, $\|p\| = \sqrt{5}$, $\|x\| = \sqrt{3}$, and $C = 3/(\sqrt{5} \cdot \sqrt{3}) \approx 0.775$. With normalized scores $C = 0.775$, $F = 0.60$, $P = 0.80$, $X = 0.50$ and weights (0.45, 0.25, 0.20, 0.10), $S = 0.34875 + 0.15 + 0.16 + 0.05 = 0.70875$. If F is unavailable, the remaining weights 0.45, 0.20, 0.10 sum to 0.75 and become 0.60, 0.267, 0.133, giving $S \approx 0.745$. These values only illustrate the arithmetic.

E. Algorithm and Complexity

Algorithm 1. Hybrid recommendation for one request

1: Validate the request and the catalog records.
2: Build the profile p_u from H_u (events before t_0).
3: Determine component availability a_k (Table I).
4: Generate candidates from the available sources; remove unavailable, excluded, duplicate, and already-seen items.
5: Compute the available scores and normalize each over the candidates.
6: Compute the effective weights (6) and the score $S(u,i)$ (7).
7: Sort by descending score; break ties by product identifier.
8: Return the first K items and log the impression.

For N eligible items and d-dimensional vectors, scoring one request costs $O(Nd)$ for the content score; the collaborative score adds a cost that depends on the neighborhood size. Candidate retrieval and caching reduce online work in larger catalogs.

VI. EXPERIMENTAL EVALUATION

The framework is evaluated in a controlled simulation so that the pipeline, the chronological protocol, and the fallback rules can be tested end to end under known conditions. A simulation cannot establish performance on a real store; Section VII discusses this limitation and evaluation on real data is left to future work.

A. Simulated Environment

The generator creates 600 items in 10 categories and 20 brands with 16-dimensional latent attributes, and 2,000 users over a 100-day horizon. Each user favors two categories; 40% of users switch to two different categories at a random day between 50 and 75 (preference drift). Item selection probability is proportional to a Zipf-like popularity term times a user–item affinity. Event types (view, cart, purchase) depend on affinity. Seventy percent of users are heavy (about 60 events) and 30% are light (about 8 events), and 15% are new users whose events all occur in the last 20 days. Five percent of items are unavailable at serving time.

B. Protocol

Events before day 80 form the training period and events in days 80–100 the test period. For each user, relevant items are those carted or purchased in the test period that the user had not interacted with in training; users with at least one relevant item are evaluated. All methods share the same split, candidate eligibility (available and unseen items), and $K = 10$ [16]. Hybrid weights are tuned by grid search (step 0.1 over the simplex, 286 combinations) on a validation window (training before day 65, evaluation in days 65–80) using Recall@10 of users with at least three events, and then fixed for the test window. Metrics are Precision@K, Recall@K, and Average Precision at K:

$$\text{Precision@}K(u) = \frac{|L_K(u) \cap R_u|}{K} \tag{8}$$

$$\text{Recall@}K(u) = \frac{|L_K(u) \cap R_u|}{|R_u|} \tag{9}$$

$$\text{AP@}K(u) = \frac{1}{\min(K, |R_u|)} \sum_{k=1}^{K} \text{Precision@}k(u) \, \text{rel}(u,k) \tag{10}$$

where R_u is the relevant set of user u and $\text{rel}(u,k) = 1$ if the item at rank k is relevant. MAP@K is the mean of AP@K over evaluated users. Catalog coverage is the number of distinct recommended items divided by the number of eligible catalog items. Every experiment is repeated for five random seeds (0–4), and results are reported as mean ± sample standard deviation over seeds.

C. Methods and Parameters

Four methods are compared under identical eligibility rules: popularity-only, content-only (3), item-based collaborative filtering (5), and the hybrid (7). Users with no training events receive the popularity fallback under every method, because no other component is defined for them. Table II lists the parameters.

TABLE II
EXPERIMENTAL PARAMETERS

Parameter	Value
Items / users	600 / 2,000
Horizon; train / test	100 days; days 0–80 / 80–100
Validation window	train < day 65; eval days 65–80
Event weights a_{type}	view 1, cart 3, purchase 5
Decay rate λ	0.02 day^{-1}
List size K	10
Collaborative minimum history	3 events
Weight grid	step 0.1 on the simplex (286)
Random seeds	0, 1, 2, 3, 4

D. Overall Results

Table III and Fig. 2 report the results for all evaluated users (about 1,668 per run). The hybrid and item-based filtering clearly outperform popularity and content-only ranking: the hybrid's Recall@10 of 0.174 ± 0.007 is 1.9 times that of popularity (0.090). The difference between the hybrid and item-based filtering is small: Recall@10 differs by 0.0014 ± 0.0026 (mean ± s.d. of paired differences), positive in 3 of 5 runs, and MAP@10 is lower by 0.0019 ± 0.0092. With five runs, no statistical significance is claimed. The tuned weights averaged $w_C = 0.06 \pm 0.09$, $w_F = 0.46 \pm 0.17$, $w_P = 0.36 \pm 0.13$, and $w_X = 0.12 \pm 0.08$, so the tuned hybrid relies mainly on collaborative and popularity evidence.

TABLE III
RESULTS FOR ALL EVALUATED USERS (MEAN ± S.D., 5 RUNS)

Method	P@10	R@10	MAP@10	Coverage
Popularity	0.047±0.002	0.090±0.006	0.041±0.003	0.040
Content	0.043±0.001	0.080±0.004	0.034±0.002	0.962
Item-CF	0.070±0.002	0.172±0.006	0.081±0.004	0.686
Hybrid	0.071±0.002	0.174±0.007	0.079±0.009	0.598

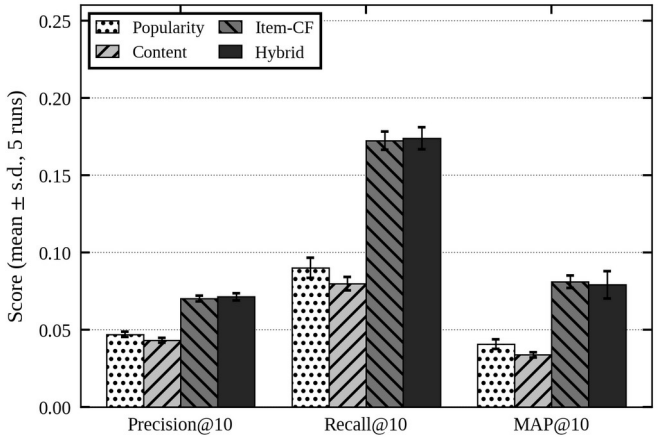


Fig. 2. Precision@10, Recall@10, and MAP@10 for all evaluated users (error bars: standard deviation over five runs).

Popularity recommends only 4% of the eligible catalog (coverage 0.040), against 0.686 for item-based filtering and 0.598 for the hybrid, which illustrates the exposure concentration of popularity ranking. Content-only ranking covers 0.962 of the catalog but has the lowest accuracy among personalized methods; a likely reason is that its profile mixes interests from before and after a drift and its features are noisy proxies of the latent attributes, although this explanation was not isolated experimentally.

E. Results by User Cohort

TABLE IV
RECALL@10 BY COHORT (MEAN ± S.D., 5 RUNS; N = USERS PER RUN)

Cohort	Popularity	Content	Item-CF	Hybrid
New (297)	0.118±0.005	0.118±0.005	0.118±0.005	0.118±0.005
Sparse (54)	0.188±0.086	0.043±0.024	0.173±0.065	0.200±0.060
Returning (1,317)	0.080±0.007	0.073±0.006	0.185±0.005	0.185±0.008
All (1,668)	0.090±0.006	0.080±0.004	0.172±0.006	0.174±0.007

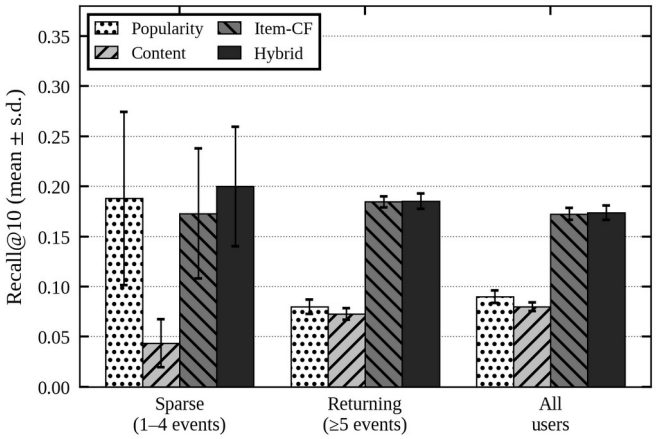


Fig. 3. Recall@10 by user cohort (error bars: standard deviation over five runs). New users are omitted because all methods use the same fallback.

For new users, all methods return the same list by design (0.118 ± 0.005), which confirms that the fallback is applied consistently. For returning users, item-based filtering and the hybrid are about 2.3 times better than popularity. Sparse-history users (about 54 per run) show the largest variance: the hybrid has the highest mean (0.200 ± 0.060), but popularity is also strong (0.188 ± 0.086) and the hybrid–CF difference of 0.027 ± 0.048 is positive in only 3 of 5 runs. This cohort is too small for firm conclusions. Content-only ranking is weakest for these users (0.043 ± 0.024), since one or two events give an unreliable profile.

F. Effect of Time Decay

Fig. 4 varies λ for returning users, re-tuning the hybrid weights for each value on a coarser grid (step 0.25); absolute values therefore differ slightly from Table III. Recall@10 of content-only ranking rises from 0.064 at λ = 0 to 0.093 at λ = 0.1. The hybrid is highest at λ = 0.05 (0.186) compared with 0.163 without decay, and drops at λ = 0.1 (0.170). This pattern is expected because the simulation contains preference drift; in a real store, λ must be selected on a validation window and may differ by category.

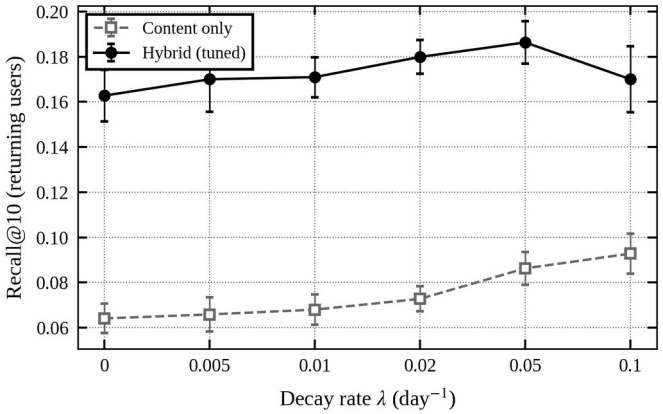


Fig. 4. Recall@10 for returning users versus decay rate λ (error bars: standard deviation over five runs).

VII. Discussion and Limitations

Answers to the research questions. (RQ1) Metadata and interaction evidence are combined through normalized component scores whose weights are tuned on a validation window; in the simulation, collaborative and popularity evidence receive most of the weight. (RQ2) A missing component is excluded and the remaining weights are renormalized per request, with popularity as the last resort, so new users receive a consistent fallback. (RQ3) The evidence needed is a chronological split, identical candidate eligibility across methods, several runs, and cohort reporting; in the simulation these show that the hybrid clearly beats popularity and content-only ranking but is not clearly different from item-based filtering overall.

Limitations. First, all data are synthetic: the generator encodes assumptions about user behavior, so absolute metric values are not transferable, and the ranking of methods may change on real data. Second, every item had at least five training events, so item cold start was not tested; the content-based fallback for new items remains unevaluated. Third, only item-based filtering and a TF–IDF-style content model were compared; matrix factorization, Bayesian personalized ranking, and learned rankers [5], [7], [12] were not. Fourth, five runs are insufficient for significance tests, and the sparse cohort is small. Fifth, relevance is a held-out proxy, and click-through or conversion effects require an instrumented online experiment. Sixth, popularity bias, fairness across brands or sellers, and privacy require further study, including diversity-aware reranking.

Future work. The next step is to replace the simulator with a documented public or store dataset, add the missing baselines, evaluate new-item cohorts, report confidence intervals, measure serving latency under larger catalogs, and run a controlled online test.

VIII. Conclusion

This paper presented a hybrid e-commerce recommendation framework that combines content similarity, item-based collaborative evidence, time-decayed popularity, and context, with per-request weight renormalization and deterministic eligibility filtering. In a five-seed simulation with a chronological protocol, the hybrid reached Recall@10 of 0.174 ± 0.007, about 1.9 times that of popularity and not clearly different from item-based filtering, while handling new and sparse-history users through an explicit fallback. These results validate the design and the evaluation procedure; establishing real-world benefit requires the evaluation on documented real data outlined above.

References

[1] P. Malekpour Alamdari, N. Jafari Navimipour, M. Hosseinzadeh, A. A. Safaei, and A. Darwesh, "A systematic study on the recommender systems in the e-commerce," *IEEE Access*, vol. 8, pp. 115694–115716, 2020, doi: 10.1109/ACCESS.2020.3002803.

[2] A. Da'u and N. Salim, "Recommendation system based on deep learning methods: A systematic review and new directions," *Artif. Intell. Rev.*, vol. 53, pp. 2709–2748, 2020, doi: 10.1007/s10462-019-09744-1.

[3] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Comput.*, vol. 7, no. 1, pp. 76–80, Jan./Feb. 2003.

[4] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. Conf. World Wide Web (WWW)*, Hong Kong, 2001, pp. 285–295.

[5] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.

[6] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 263–272.

[7] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. 25th Conf. Uncertainty Artif. Intell. (UAI)*, Montreal, QC, Canada, 2009, pp. 452–461.

[8] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Model. User-Adapt. Interact.*, vol. 12, no. 4, pp. 331–370, 2002.

[9] N. AlRossais, D. Kudenko, and T. Yuan, "Improving cold-start recommendations using item-based stereotypes," *User Model. User-Adapt. Interact.*, vol. 31, pp. 867–905, 2021, doi: 10.1007/s11257-021-09293-9.

[10] C. Bernardis and P. Cremonesi, "NFC: A deep and hybrid item-based model for item cold-start recommendation," *User Model. User-Adapt. Interact.*, vol. 32, pp. 747–780, 2022, doi: 10.1007/s11257-021-09303-w.

[11] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. 10th ACM Conf. Recommender Syst. (RecSys)*, Boston, MA, USA, 2016, pp. 191–198.

[12] D.-N. Nguyen, V.-H. Nguyen, T. Trinh, T. Ho, and H.-S. Le, "A personalized product recommendation model in e-commerce based on retrieval strategy," *J. Open Innov., Technol., Market, Complexity*, vol. 10, no. 2, Art. no. 100303, 2024, doi: 10.1016/j.joitmc.2024.100303.

[13] A. Valencia-Arias *et al.*, "Artificial intelligence and recommender systems in e-commerce: Trends and research agenda," *Intell. Syst. Appl.*, vol. 24, Art. no. 200435, 2024, doi: 10.1016/j.iswa.2024.200435.

[14] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[15] Y. Ding and X. Li, "Time weight collaborative filtering," in *Proc. 14th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, Bremen, Germany, 2005, pp. 485–492.

[16] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "Evaluating collaborative filtering recommender systems," *ACM Trans. Inf. Syst.*, vol. 22, no. 1, pp. 5–53, Jan. 2004.